

1. Opis projektu

Celem projektu jest stworzenie aplikacji webowej wykorzystującej modele generatywnej sztucznej inteligencji (LLM) do tworzenia spersonalizowanych ścieżek edukacyjnych dla programistów. System generuje materiały dydaktyczne dopasowane do technologii wybranej przez użytkownika (np. Java, Spring Boot, React, Python), a także do jego preferencji, takich jak poziom zaawansowania, preferowany sposób nauki (teoria/praktyka/wideo), dostępny czas nauki oraz tempo przyswajania wiedzy.

Aplikacja dodatkowo zbiera feedback od użytkowników, który jest wykorzystywany do dynamicznego dostosowywania kolejnych materiałów edukacyjnych. W ramach projektu planuje się przygotowanie 3–4 różnych ścieżek edukacyjnych generowanych przez model AI.

2. Technologie, narzędzia i biblioteki

2.1 Frontend – React + Next.js

Uzasadnienie wyboru:

React jest jedną z najpopularniejszych bibliotek do budowy interfejsów użytkownika.

Umożliwia tworzenie dynamicznych, komponentowych aplikacji webowych, co jest szczególnie ważne w systemie edukacyjnym, gdzie treści są generowane i aktualizowane w czasie rzeczywistym.

Next.js rozszerza możliwości React o renderowanie po stronie serwera (SSR) oraz generowanie statyczne (SSG), co poprawia wydajność aplikacji oraz jej indeksowanie przez wyszukiwarki.

Zastosowanie:

- interfejs użytkownika kursanta,
- wybór preferencji edukacyjnych,
- prezentacja wygenerowanych ścieżek nauki,
- system ocen i opinii,
- dynamiczne odświeżanie treści generowanych przez AI.

2.2 Backend – Python (FastAPI)

Uzasadnienie wyboru:

Python jest standardem w obszarze sztucznej inteligencji, analizy danych oraz integracji z modelami LLM. Dzięki ogromnemu ekosystemowi bibliotek (OpenAI SDK, LangChain, LlamaIndex, NumPy, Pandas) idealnie nadaje się do budowy systemu generowania treści edukacyjnych.

FastAPI zapewnia wysoką wydajność, asynchroniczność oraz automatyczną dokumentację API (Swagger/OpenAPI), co znacząco przyspiesza rozwój i testowanie aplikacji.

Zastosowanie:

- generowanie ścieżek edukacyjnych przy użyciu LLM,
- przetwarzanie preferencji użytkownika,
- komunikacja z API modeli AI,
- implementacja REST API dla frontendu,
- obsługa systemu feedbacku,
- personalizacja materiałów edukacyjnych,
- integracja z bazą danych,
- implementacja RAG (Retrieval-Augmented Generation).

2.3 Model generatywny AI – OpenAI API

Uzasadnienie wyboru:

Modele LLM umożliwiają generowanie wysokiej jakości treści edukacyjnych w sposób dynamiczny i kontekstowy. Dzięki możliwości dostosowania stylu odpowiedzi oraz poziomu trudności, mogą one pełnić rolę inteligentnego tutora.

Zastosowanie:

- generowanie kursów programistycznych,
- tworzenie ćwiczeń i quizów,

- adaptacja materiałów do poziomu użytkownika,
 - wyjaśnianie zagadnień programistycznych,
 - modyfikacja ścieżki nauki na podstawie feedbacku.
-

2.4 Orkiestracja LLM – LangChain / LlamaIndex

Uzasadnienie wyboru:

LangChain i LlamaIndex upraszczają tworzenie aplikacji opartych o LLM poprzez zarządzanie promptami, pamięcią konwersacji oraz integrację z bazami wiedzy. Umożliwiają również implementację systemów RAG, co zwiększa dokładność generowanych odpowiedzi.

Zastosowanie:

- budowa pipeline generowania kursów,
 - zarządzanie promptami edukacyjnymi,
 - integracja z dokumentacją technologiczną,
 - pamięć kontekstu użytkownika,
 - wyszukiwanie semantyczne materiałów.
-

2.5 Baza danych – PostgreSQL + pgvector

Uzasadnienie wyboru:

PostgreSQL jest stabilną i wydajną relacyjną bazą danych. Rozszerzenie pgvector umożliwia przechowywanie embeddingów, co pozwala na semantyczne wyszukiwanie materiałów edukacyjnych.

Zastosowanie:

- przechowywanie profili użytkowników,
- zapisywanie historii nauki,
- przechowywanie wygenerowanych kursów,
- analiza feedbacku,

- system rekomendacji materiałów,
 - wyszukiwanie podobnych treści.
-

2.6 Autoryzacja – Keycloak / Auth0

Uzasadnienie wyboru:

Systemy te zapewniają bezpieczne zarządzanie użytkownikami oraz gotowe mechanizmy logowania i autoryzacji. Keycloak jest rozwiązaniem open-source, natomiast Auth0 oferuje prostszą integrację.

Zastosowanie:

- logowanie i rejestracja użytkowników,
 - zarządzanie rolami (kursant, admin),
 - ochrona danych użytkowników,
 - integracja z profilem edukacyjnym.
-

2.7 Konteneryzacja – Docker + Docker Compose

Uzasadnienie wyboru:

Docker umożliwia łatwe uruchamianie całego systemu w izolowanych środowiskach, co ułatwia testowanie i wdrażanie aplikacji.

Zastosowanie:

- uruchamianie frontend + backend + baza danych,
 - izolacja usług,
 - łatwe wdrożenie,
 - skalowanie systemu.
-

2.8 Testowanie – Pytest + Cypress

Uzasadnienie wyboru:

Pytest jest standardem testowania aplikacji Python, natomiast Cypress umożliwia testy end-to-end w przeglądarce.

Zastosowanie:

- testy backendu (API FastAPI),
 - testy generowania ścieżek edukacyjnych,
 - testy integracyjne z LLM,
 - testy UI (frontend),
 - symulacja zachowań użytkownika.
-

3. Ścieżki edukacyjne (generowane przez AI)

System generuje 3–4 warianty ścieżek:

1. Java Backend Developer (Spring Boot)

- Java podstawy, OOP
- Spring Boot REST API
- bazy danych i Hibernate

2. Frontend Developer (React)

- HTML, CSS, JavaScript
- React i komponenty
- zarządzanie stanem (Redux/Zustand)

3. Fullstack JavaScript Developer

- Node.js + Express
- React frontend
- integracja fullstack

4. AI for Developers (Python + LLM)

- Python podstawy
 - prompt engineering
 - integracja OpenAI API
 - RAG i embeddingi
-

4. Podsumowanie

Zaproponowany system łączy nowoczesne technologie webowe oraz modele generatywnej sztucznej inteligencji w celu stworzenia inteligentnej platformy edukacyjnej. Frontend oparty o React i Next.js zapewnia interaktywność i responsywność, backend w Pythonie (FastAPI) umożliwia efektywną integrację z modelami LLM, a PostgreSQL z pgvector pozwala na przechowywanie i analizę danych semantycznych.

Zastosowanie modeli AI umożliwia dynamiczne generowanie spersonalizowanych ścieżek edukacyjnych, co znacząco zwiększa efektywność nauki oraz dopasowanie materiałów do indywidualnych potrzeb użytkownika.